

Documentación Técnica del Sistema Multi-Agente “Sesame”

Índice

1. Arquitectura General
2. Componentes
 - 2.1. Frontend
 - 2.2. API Server
 - 2.3. MCP Server
 - 2.4. Agentes Especializados
3. Grafo de Nodos (LangGraph)
4. Integración con MCP
5. Docker
6. Monitoreo: Prometheus & Grafana
7. CI/CD (GitHub Actions)
8. Configuración de VSCode
9. Pruebas unitarias

1. Arquitectura General

1.1 API Server

1.1.1 Arquitectura y Capas

1. Hexagonal

- El núcleo de la aplicación (orquestración de agentes) está aislado de frameworks externos (FastAPI, Prometheus).
- **Orchestrator** y **AgentGraph** actúan como “dominio” puro.
- FastAPI, pydantic, middleware y logging son adaptadores que exponen puertos de entrada/salida.

2. Capas

- **Entrada (Controllers / Routers):**
 - main.py define rutas (/health, /mcp/status, /api/v1/query, /metrics).
 - Validación y serialización con: Pydantic (QueryRequest, HealthResponse, etc.).
- **Aplicación (Orquestador):**
 - Orchestrator.process_query() encapsula la lógica de negocio de enrutamiento.
- **Dominio (Graph & Agentes):**
 - AgentGraph (pattern State Machine) define el flujo de trabajo.
 - Agentes especializados (FinanceAgent, MarketingAgent, ...) implementan la lógica de cada dominio.
- **Infraestructura:**
 - Integración con MCP (mcp_integration.py).
 - Persistencia de logs, métricas Prometheus y conexiones HTTP.

1.1.2 Patrones de Diseño

- **Chain of Responsibility / State Machine**

- AgentGraph usa LangGraph para modelar un grafo de nodos y transiciones condicionales (Guardrail → Router → Agentes → Summary → END).

- **Strategy**

- Cada agente implementa su propia estrategia de procesamiento (`_execute_impl`).
- El router decide qué estrategia (agente) usar según contexto y palabras clave.

- **Singleton / Lazy Initialization**

- Cliente MCP (`_mcp_client`) se inicializa una sola vez y se reutiliza.

- **Factory**

- `AgentGraph._build_graph()` arma dinámicamente nodos y aristas del grafo.

- **Middleware**

- `PrometheusMiddleware` encapsula la lógica transversal de métricas (latencia, conteo).

- **Adapter**

- Adaptación de herramientas MCP al formato esperado por agentes (closures que envuelven llamadas al servidor MCP).

1.2. Frontend

1.2.1 Arquitectura

- **Single Page Application con Vue.js + Vite**
 - Enrutamiento en el cliente, componentes y hooks.
 - Peticiones fetch/axios a la API Server en <http://localhost:8000> y <http://localhost:4000>.
- **Estrategia de Build & Deploy**
- **Multi-Stage Docker:**
 - Build: node:18-alpine, npm ci, npm run build.
 - Prod: nginx:alpine sirve carpeta dist.
- **Estructura de Carpetas**
 - src/: componentes, páginas, servicios API.
 - public/: assets estáticos.
 - index.html, tailwind.config.js, vite.config.js.

1.2.2 Patrones de Diseño

- **Component-Based Architecture**
- UI dividida en componentes reutilizables (Botones, Formularios, Tablas...).
- **Unidirectional Data Flow**
- Estado gestionado localmente o con Context API / hooks personalizados.
- Props → Componentes Hijos → Eventos de callback.
- **Environment Configuration**
- Variables de entorno (.env) desacoplan URL de backend.
- **Cache / Lazy Loading**
- Lazy loading de rutas o componentes pesados (si se implementa).

1.3. MCP Server

1.3.1 Arquitectura

- **Microservicio Ligero**
 - Basado en FastAPI; expone protocolo MCP para listar e invocar herramientas.
 - Carpeta app/tools/ contiene cada herramienta como módulo independiente.
- **Plugin-Like / Registry**
 - Al iniciar, main.py o un componente de arranque escanea app/tools/ y registra dinámicamente las herramientas disponibles.
 - Documento de metadatos (name, description, args_schema).
- **Endpoints Principales**
 - GET /status: estado general y número de herramientas.
 - GET /tools : lista detallada de herramientas y esquemas.
 - POST /tool/{name}: invoca la función asociada con payload JSON.

1.3.2 Patrones de Diseño

- **Registry / Plugin**
 - Las herramientas se registran en un diccionario dinámico y se invocan por nombre.
- **Adapter**
 - Cada herramienta adapta su interfaz interna a un contrato MCP unificado: recibe dict → retorna dict.
- **Healthcheck**
 - Endpoint /status sirve de healthcheck
- **Separation of Concerns**
 - Lógica de herramientas separada de la lógica de transporte HTTP.

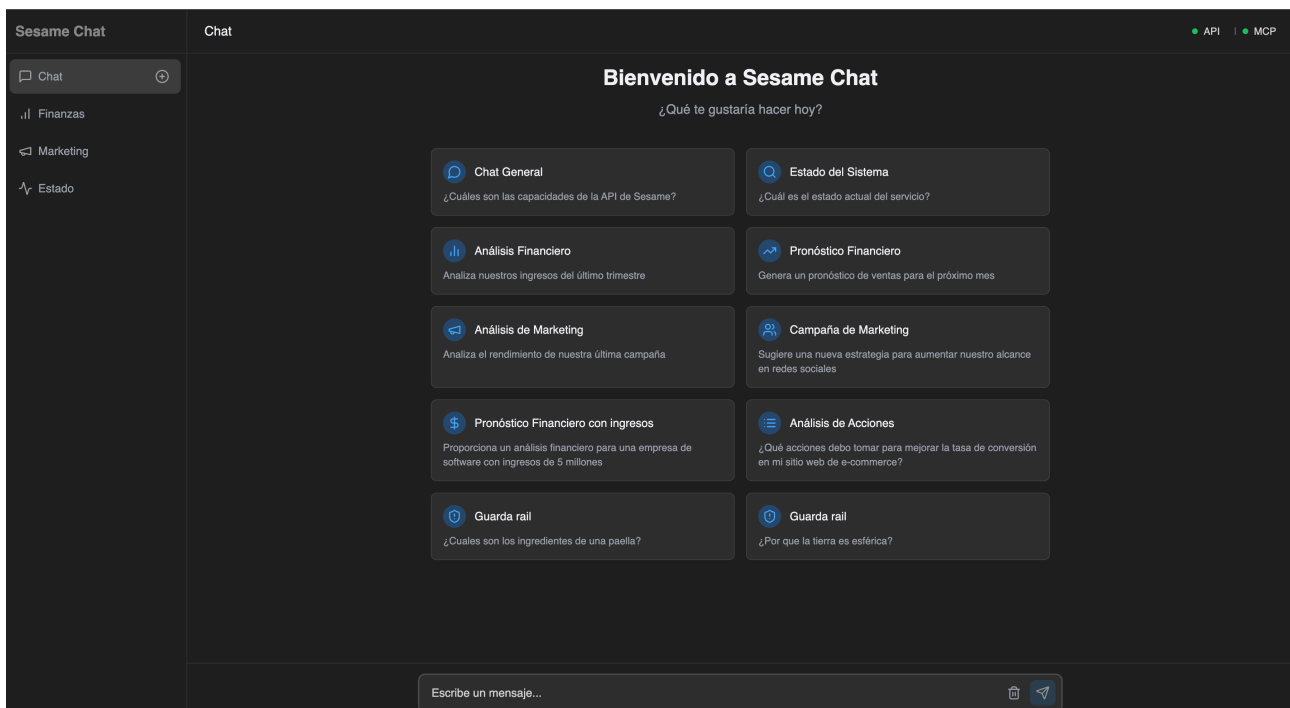
1.4. Comunicación Entre Componentes

- **Front → API:**
 - Peticiones REST a /api/v1/query y endpoints de estado.
- **API → MCP:**
 - Cliente HTTP o STDIO (MCPClient) consume /status, /tools, /tool/{name}.
- **Docker Compose:**
 - Red interna permite referirse a servicios por nombre (api_server, mcp_server).
 - depends_on y healthchecks garantizan arranque y orden correcto.

2. Componentes

2.1 Frontend (frontend/)

- Tecnologías: Vite + Vue.js + Tailwind CSS.



2.2 API Server (api_server/)

- main.py:
 - **Configura logger** y directorio logs/.
 - **Define métricas Prometheus:** counters, histograms, gauges.
 - **Middleware PrometheusMiddleware** mide latencia y conexiones.
 - **Endpoints:**
 - / Bienvenida y link a Swagger.
 - /health (HealthResponse): estado API + conexión MCP.
 - /mcp/status (MCPStatusResponse): estado detallado de herramientas MCP.
 - /metrics: expone métricas para Prometheus.
 - /api/v1/query: recibe peticiones estilo chat, enruta a agentes. pudiendo sugerir agentes con un parámetro en la llamada.
 - **Orquestador** (app/core/orchestrator.py):
 - process_query() genera request_id, invoca process_request().
 - process_request() mide tiempo, ejecuta AgentGraph.run(), construye respuesta.
 - En errores, registra con MetricsCollector.record_error().
 - **Configuración** (app/core/config.py): Pydantic lee .env, define tiempo de timeout, API keys.
- **Logs:** carpeta app/logs/ montada en Docker.

2.3 MCP Server (mcp_server/)

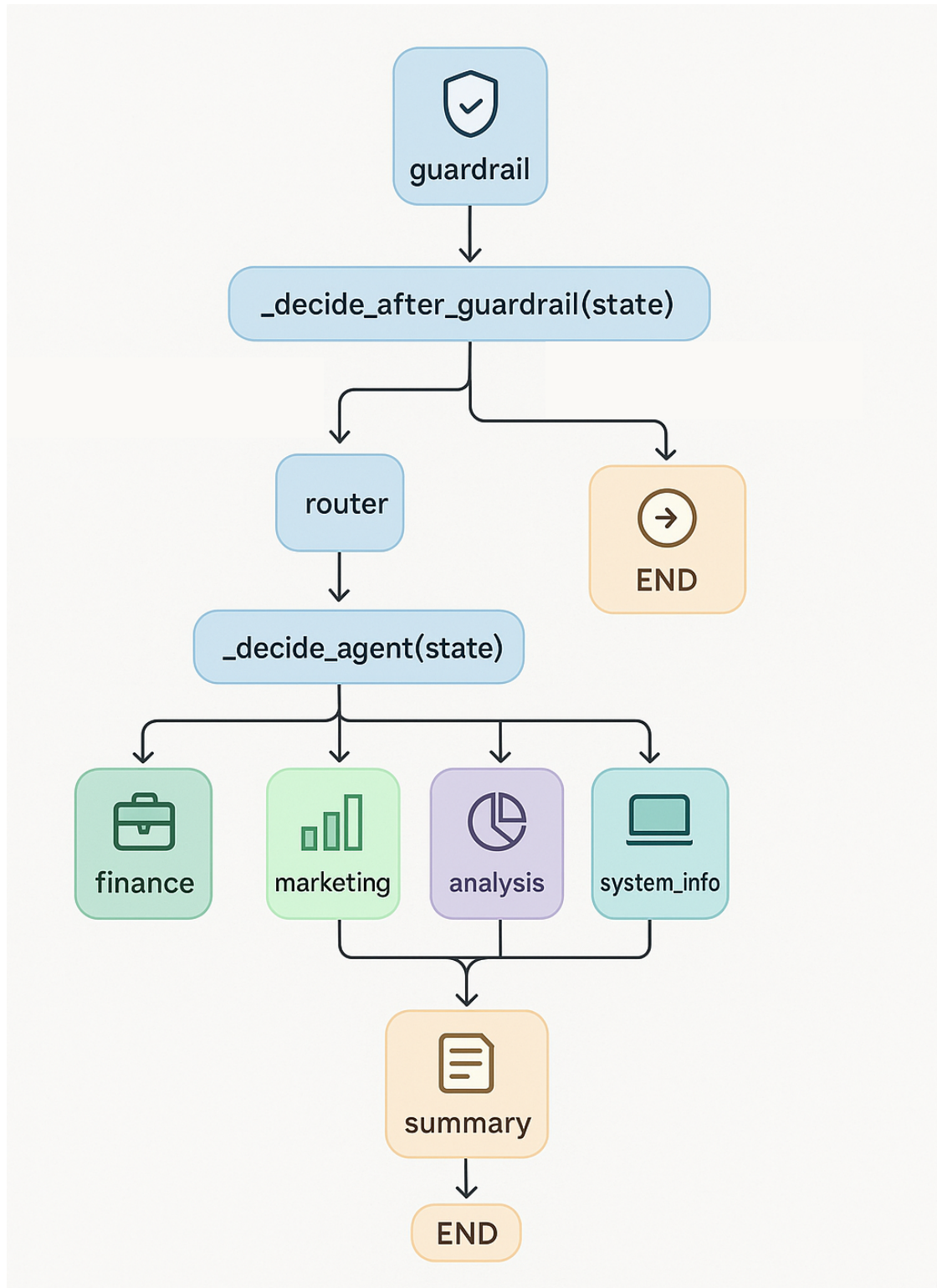
- main.py:
 - **FastAPI.**
 - **Endpoints:**
 - GET /status: informa estado y lista de herramientas.
 - GET /tools: devuelve metadatos de herramientas.
 - POST /tool/{name}: invoca herramienta con JSON de argumentos.
 - **Healthcheck** en /status.

2.4 Agentes Especializados (app/agents/)

Cada agente extiende BaseAgent (base.py) y define `_execute_impl()`.

Agente	Archivo	Responsabilidad
GuardrailAgent	guardrail_agent.py	Filtra consultas fuera de scope.
RouterAgent	router_agent.py	Clasifica y enruta consultas.
FinanceAgent	finance_agent.py	Análisis financiero y métricas.
MarketingAgent	marketing_agent.py	Análisis de marketing y KPI.
AnalysisAgent	analysis_agent.py	Análisis de datos y extracción de insights.
SystemInfoAgent	system_info_agent.py	Estado y capacidades del sistema.
SummaryAgent	summary_agent.py	Síntesis y enriquecimiento de respuestas.
OperationsAgent	operations_agent.py	Llamadas API externas y workflows.
DataLookupAgent	data_lookup_agent.py	Búsqueda y consulta de datos externos.
ActionAgent	action_agent.py	Ejecuta acciones o triggers.

3. Grafo de Nodos (LangGraph)



3.1 Nodos y Funciones

Nodo	Función interna	Descripción
guardrail	<code>_check_guardrail</code>	Ejecuta GuardrailAgent, marca <code>in_scope</code> o error.
router	<code>_route</code>	Ejecuta RouterAgent, actualiza <code>current_agent</code> .
finance	<code>_process_finance</code>	Llama a <code>FinanceAgent.execute()</code> , guarda <code>raw_response</code> .
marketing	<code>_process_marketing</code>	Lógica similar a finance.
analysis	<code>_process_analysis</code>	Ejecuta AnalysisAgent.
system_info	<code>_process_system_info</code>	Ejecuta SystemInfoAgent.
summary	<code>_process_summary</code>	Ejecuta SummaryAgent para síntesis final.
END	—	Nodo terminal, retorna resultado global.

3.2 Transiciones

- Desde guardrail → `_decide_after_guardrail(state)`:
 - Si está fuera del scope → END.
 - Si está en el scope → router.
- Desde router → `_decide_agent(state)`:
 - Mapea `current_agent` a nodos: finance, marketing, analysis, system_info.
- Especializados (finance, marketing, etc.) → summary.
- summary → END.

4. Integración con MCP

- **Cliente MCP** (app/agents/mcp_integration.py):
 - `get_mcp_tools_sync()`: inicializa MCPClient, lista herramientas.
 - `configure_agent_with_mcp(agent, tools)`: inyecta herramientas en `agent.tools`.
 - `get_mcp_client()`: devuelve cliente HTTP.
- **Flujo:**
 - Al iniciar agente, invocar `get_mcp_tools_sync()`.
 - `configure_agent_with_mcp(...)`.
 - Durante `_execute_impl`, llama a closures que envían petición a MCP.

5. Docker

5.1 Dockerfiles

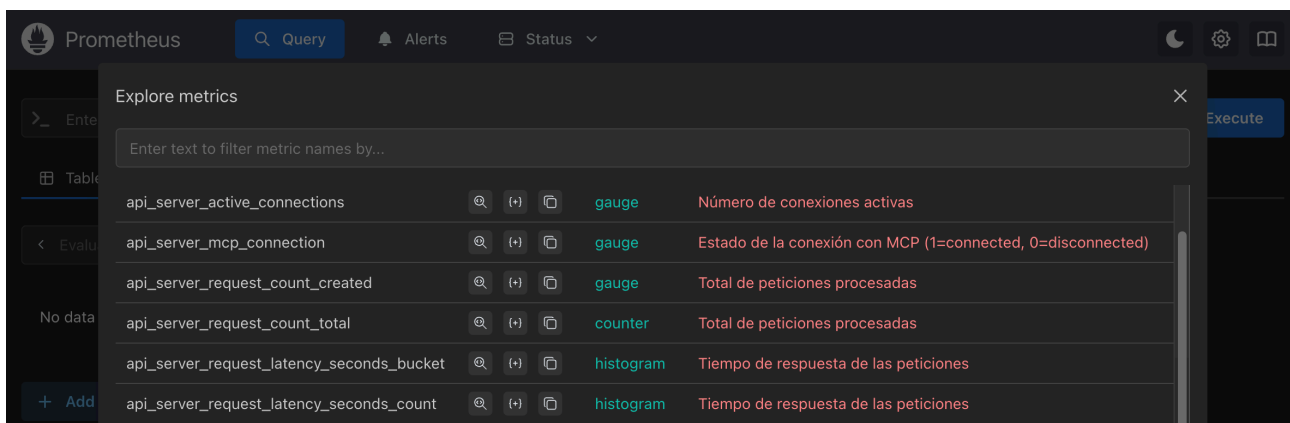
- **API Server** (api_server/Dockerfile): Python 3.10-slim, instala deps, crea logs, expone 8000.
- **MCP Server** (mcp_server/Dockerfile): Python 3.10-slim, instala deps, crea logs, expone, expone 4000.
- **Frontend** (frontend/Dockerfile): build en Node, producción en Nginx.

5.2 Docker Compose

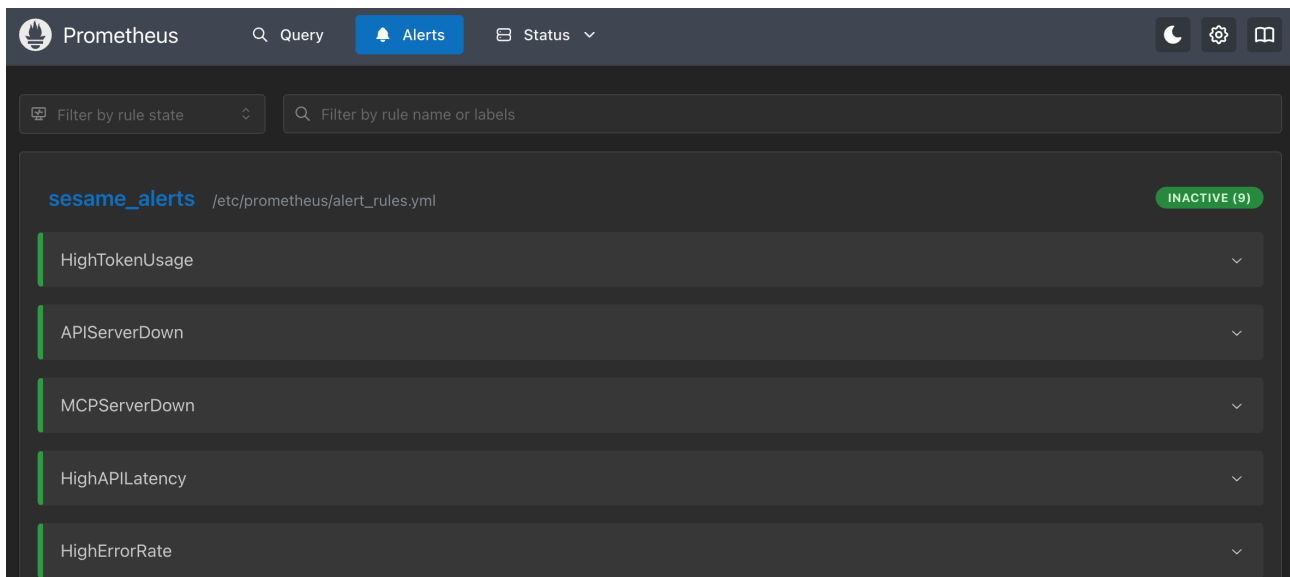
```
docker-compose.yml
version: '3.8'
services:
  mcp_server:
    build: ./mcp_server
    ports: ["4000:4000"]
    healthcheck: curl -f http://localhost:4000/status
  api_server:
    build: ./api_server
    ports: ["8000:8000"]
    depends_on: [mcp_server]
    healthcheck: curl -f http://localhost:8000/health
  frontend:
    build: ./frontend
    ports: ["3000:80"]
    depends_on: [api_server]
    healthcheck: wget -qO- http://localhost:80
  prometheus:
    image: prom/prometheus:latest
    ports: ["9090:9090"]
    volumes:
      - ./prometheus:/etc/prometheus
  grafana:
    image: grafana/grafana:latest
    ports: ["3001:3000"]
    volumes:
      - ./grafana/dashboards:/var/lib/grafana/dashboards
      - ./grafana/provisioning:/etc/grafana/provisioning
  node-exporter:
    image: prom/node-exporter:latest
    ports: ["9100:9100"]
  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    ports: ["8080:8080"]
networks:
  default: { driver: bridge }
volumes:
  prometheus_data: {}
  grafana_data: {}
```

6. Monitoreo: Prometheus & Grafana

- **Prometheus** (prometheus/prometheus.yml):
configura scrape_configs para:
 - API Server (api_server:8000/metrics)
 - node-exporter, cAdvisor

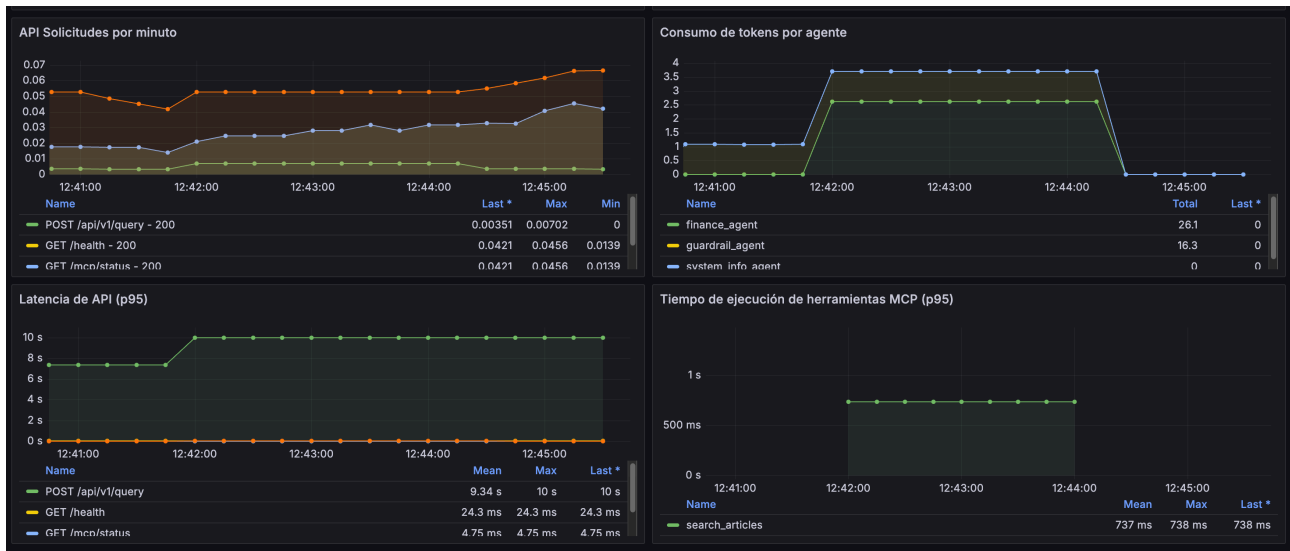


- **Alert Rules:** (prometheus/alert_rules.yml): CPU alto, latencia, errores.



• Grafana:

- Dashboards JSON en grafana/dashboards/.
- Provisioning de datasources en grafana/provisioning/.
- Plugin: grafana-piechart-panel.

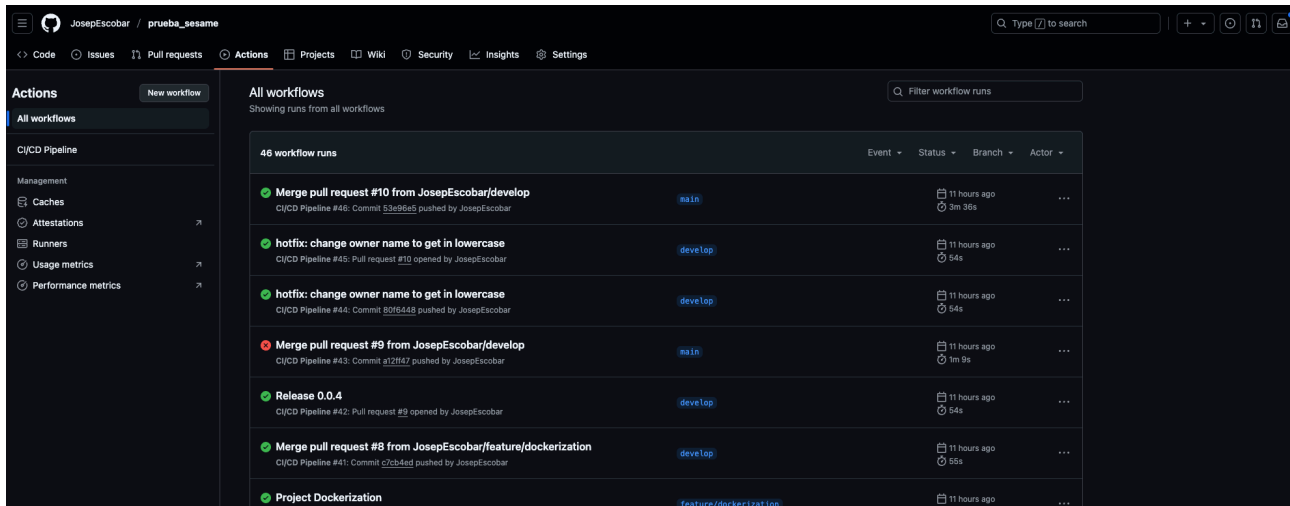


Métricas clave en API Server:

- api_server_request_count
- api_server_request_latency_seconds
- api_server_token_count
- api_server_active_connections
- api_server_status
- api_server_mcp_connection

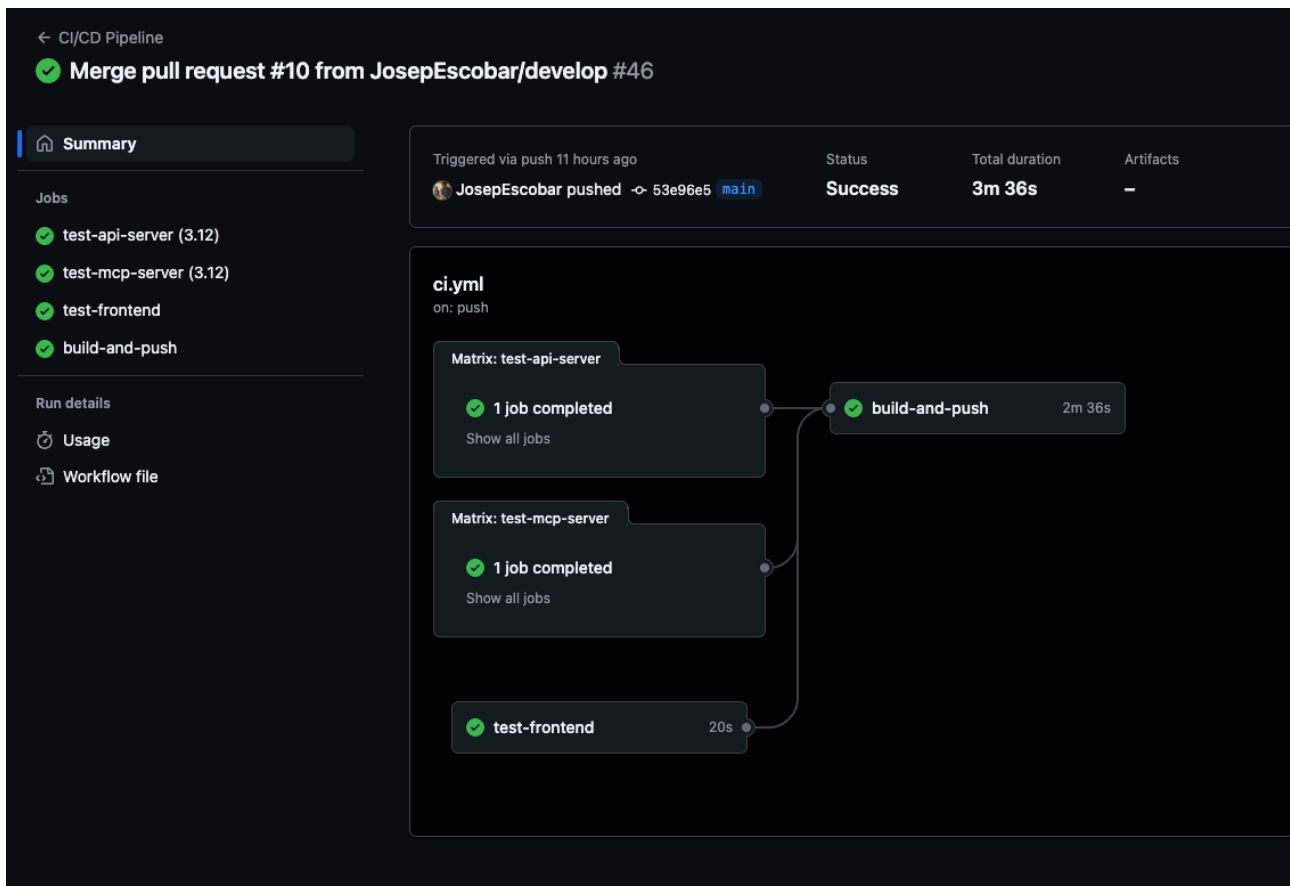
7. CI/CD (GitHub Actions)

Ejecuciones de la CI/CD:



Workflow Run	Status	Branch	Actor
Merge pull request #10 from JosepEscobar/develop	Success	main	JosepEscobar
hotfix: change owner name to get in lowercase	Success	develop	JosepEscobar
hotfix: change owner name to get in lowercase	Success	develop	JosepEscobar
Merge pull request #9 from JosepEscobar/develop	Success	main	JosepEscobar
Release 0.0.4	Success	develop	JosepEscobar
Merge pull request #8 from JosepEscobar/feature/dockerization	Success	develop	JosepEscobar
Project Dockerization	Success	feature/dockerization	JosepEscobar

Flujo completo al hacer merge a main:



Summary

Triggered via push 11 hours ago

Status: **Success**

Total duration: **3m 36s**

Artifacts: **-**

Jobs:

- test-api-server (3.12)
- test-mcp-server (3.12)
- test-frontend
- build-and-push

Run details:

- Usage
- Workflow file

ci.yml
on: push

Matrix: test-api-server

- 1 job completed

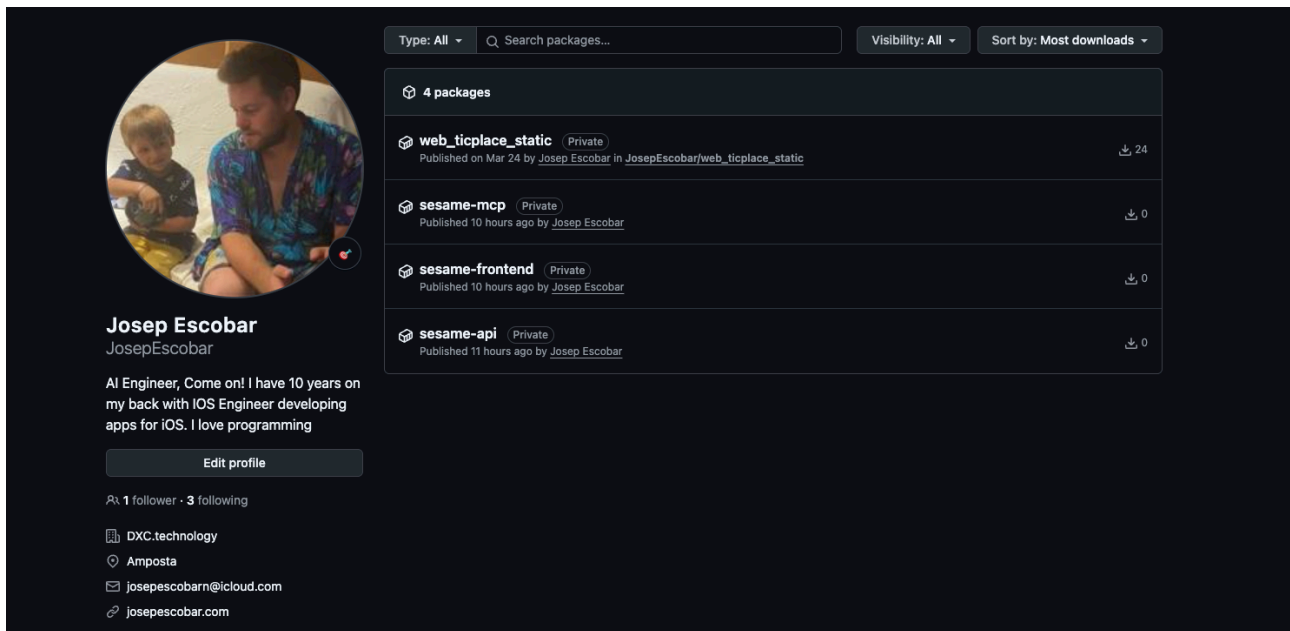
Matrix: test-mcp-server

- 1 job completed

test-frontend (20s)

build-and-push (2m 36s)

Packages publicados en GHCR:



The screenshot shows the GitHub Packages profile for Josep Escobar. On the left, there is a circular profile picture of a man with a beard and a child, and a bio stating: "AI Engineer, Come on! I have 10 years on my back with IOS Engineer developing apps for iOS. I love programming". Below the bio is an "Edit profile" button. To the right of the bio, it says "1 follower · 3 following" and lists repositories: "DXC.technology", "Amposta", "josepescobarn@icloud.com", and "josepescobar.com". On the right side, there is a list of 4 packages under the "web_ticplace_static" namespace. The packages are: "web_ticplace_static" (Private, Published on Mar 24, 24 downloads), "sesame-mcp" (Private, Published 10 hours ago, 0 downloads), "sesame-frontend" (Private, Published 10 hours ago, 0 downloads), and "sesame-api" (Private, Published 11 hours ago, 0 downloads).

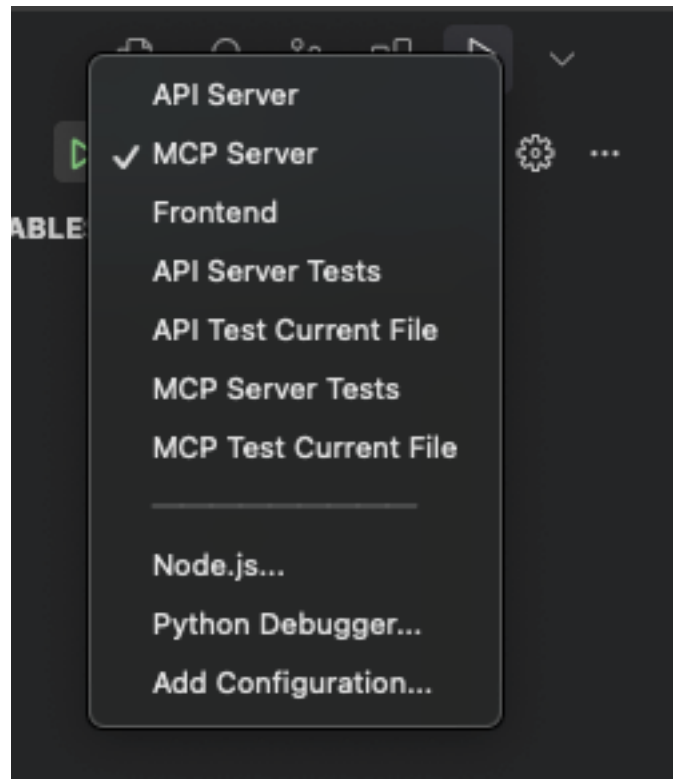
Package Name	Visibility	Published	Downloads
web_ticplace_static	Private	Published on Mar 24 by Josep Escobar in JosepEscobar/web_ticplace_static	24
sesame-mcp	Private	Published 10 hours ago by Josep Escobar	0
sesame-frontend	Private	Published 10 hours ago by Josep Escobar	0
sesame-api	Private	Published 11 hours ago by Josep Escobar	0

Archivo: **.github/workflows/ci.yml**

- **test-api-server:** Python 3.12, checkout, pip install, lint con Ruff, pytest+coverage (genera coverage.xml).
- **test-mcp-server:** Python 3.12, checkout, pip install, lint con Ruff, pytest+coverage (genera coverage.xml).
- **test-frontend:** Node.js 18, npm ci, npm run lint, npm run build.
- **build-and-push:** al hacer push en main, Buildx + GHCR, tags latest y SHA de commit.
- **Coverage en PR:** usa xml2js para parsear coverage.xml y comentar porcentaje en GitHub PR.
- **Construye y publica las imágenes en GHCR** al recibir un commit en master.

8. Configuración de VSCode

- **.vscode/settings.json:**
 - editor.formatOnSave, acciones de Ruff, python.testing.pytestArgs: rutas de tests.
 - python.analysis.extraPaths: incluye ./api_server, ./mcp_server.
- **.vscode/launch.json:**
 - Config para debug de API Server (main.py), MCP Server y Frontend (npm run dev).
 - Configs para pytest (carpeta y archivo actual).



9. Pruebas

- **API Server** (api_server/app/tests): pruebas unitarias.
- **MCP Server** (mcp_server/app/tests): pruebas unitarias.